DIM0501 — Boas Práticas de Programação

Projeto da Unidade 1 — AgendaFácil

Versão C++

Prof. Marcus Terra · DIMAp/UFRN · Semestre 2026.1

Integrantes (nome e matrícula)	Linguagem escolhida
Bianca Karolinne Souza Silva (20250031862) João Davi dos Santos Araujo (20250039243) William Ariel de Souza e Silva (20250055044)	C++

Etapa 1 — Diagnóstico

Categoria	Anomalia encontrada (e onde)	Por que é um problema
1. Adequação e lógica	-Função de remarcar checa todas as combinações possíveis com 2 for aninhados. -A função de listar os atendimentos do mês exibe apenas o atendimento com aquela data específica. A função está errada.	-Podemos apenas extrair a consulta para simplificar a lógica.
2. Manutenibilidade	-Função de agendar com try e catch, onde os erros não são mostrados. -Caso o agendamento da consulta falhe, nós não sabemos o motivo. -Caso a remarcação da consulta falhe, nós não sabemos o motivo. -Funções de agendar e remarcar mal modularizadas com mais de uma responsabilidade.	-Try e catch desnecessários, que acaba escondendo os erros. -O cliente não sabe o que ele precisa mudar para conseguir agendar ou remarcar a consulta com sucesso. -Muitas responsabilidades em uma única função tornam a manutenção do código difícil.
3. Organização e estética	-Função de remarcar com ifs aninhados.	-Ifs aninhados são confusos, é melhor usar early return.

	-Função de agendar com ifs aninhados.	
4. Programação defensiva e segurança	-No agendamento, caso o usuário digite caracteres inesperados nada será feito. -A mesma coisa acontece para remarcação.	-Resulta em comportamento inesperado nas funções.
5. Desempenho e recursos	-Função de remarcar com complexidade quadrática na checagem.	-Aumenta o tempo de execução do código e gasto de recurso computacional.

Decisões importantes tomadas

- Refatoração do código

Na implementação do módulo de agendamento, a decisão central foi modelar cada atendimento como uma struct com campos nomeados (cliente, profissional, data, horário, duração e status), em vez de estruturas acessadas por posição, o que torna o código autoexplicativo, preserva o tipo de cada dado e reduz o risco de erros ao alterar a estrutura. As regras de negócio foram separadas em funções de responsabilidade única (validar_horario, validar_duracao e validar_disponibilidade), reutilizadas tanto no agendamento quanto na remarcação, o que elimina lógica duplicada e facilita a manutenção. Adotou-se programação defensiva: nenhum dado recebido é considerado confiável, sendo verificados o intervalo de funcionamento (8h às 17h), a positividade da duração e a existência de conflito para o mesmo profissional, data e horário, com mensagens específicas para cada falha, em vez de tratamento silencioso. O fluxo das funções foi simplificado com retornos antecipados no lugar de estruturas condicionais aninhadas, o que o deixa linear e legível, e a busca por atendimentos foi isolada em uma função própria, tratando o caso de atendimento inexistente na remarcação. Por fim, foram removidos o bloco try/catch vazio antigo, que ocultava erros, e o código comentado obsoleto, além de se utilizarem parâmetros de texto por referência constante para evitar cópias desnecessárias e alterações indevidas.

- **Dívidas técnicas deixadas:**

O sistema considera os meses como tendo apenas 30 dias. A validação não aceita valores maiores que 30 para o mês. Assim, até que isso seja implementado no futuro, não será possível marcar atendimentos para meses que possuem dia 31. Além disso, não há checagem especial para o mês de fevereiro.

- **A função validar_data:**

No código, note que esta função possui um bloco try/catch com o catch capturando qualquer exceção. Tal comportamento não é visto como um problema, dado que, na situação do sistema, a única exceção possível de ser lançada é uma exceção do tipo std::invalid_argument, causada por uso de valores não numéricos, gerando justamente esse erro na conversão da string para int. Isso foi pensado tendo em vista que o foco principal é invalidar entradas indevidas inseridas pelo usuário.